



Univerzitet u Sarajevu
Elektrotehnički fakultet
Odsjek za automatiku i elektroniku



Predmet: Signali i sistemi 2

Seminarski rad

Expectation Maximization algoritam

Student:
Berina Biberović, 18402/2068

Predmetni profesor:
prof. dr Emir Sokić

Akademski: 2022/2023 godina

Sadržaj

1	Uvod	6
2	Generalizirani EM algoritam	7
2.1	Osnovna ideja	7
2.2	EM za GMM	8
3	Model sa dvije normalne raspodjele	9
3.1	Teorijska pozadina	9
3.2	Pseudokod za EM algoritam za model sa dvije normalne raspodjele	10
4	Primjer 1	12
5	Primjer 2	16
6	Zaključak	20
A	Korištene definicije i teoreme	21
B	Implementacija algoritma	25

Popis slika

4.1	Histogram Y i dobivena aproksimacija za Y u primjeru 1	13
4.2	Logaritamska vjerovatnoća za primjer 1	13
4.3	Grafik koji prikazuje promjene parametara $\hat{\mu}_1$ i $\hat{\mu}_2$ kroz iteracije za primjer 1	14
4.4	Grafik koji prikazuje promjene parametara $\hat{\sigma}_1$ i $\hat{\sigma}_2$ kroz iteracije za primjer 1	14
4.5	Grafik koji prikazuje promjenu parametra $\hat{\pi}$ kroz iteracije za primjer 1 . . .	15
5.1	Histogram Y i dobivena aproksimacija za Y u primjeru 2	16
5.2	Logaritamska vjerovatnoća za primjer 2	17
5.3	Grafik koji prikazuje promjene parametara $\hat{\mu}_1$ i $\hat{\mu}_2$ kroz iteracije za primjer 2	18
5.4	Grafik koji prikazuje promjene parametara $\hat{\sigma}_1$ i $\hat{\sigma}_2$ kroz iteracije za primjer 2	18
5.5	Grafik koji prikazuje promjenu parametra $\hat{\pi}$ kroz iteracije za primjer 2 . . .	19

Popis tablica

4.1	Podaci koji sačinjavaju Y u primjeru 1	12
-----	--	----

Sažetak

Ovaj rad obrađuje temu Expectation Maximization algoritma. U prvom dijelu je pokazana matematička pozadina uvoda u generalizirani Expectation Maximization algoritam. Nakon toga je data matematička pozadina za algoritam primijenjen na model sa dvije normalne raspodjele. Glavni fokus rada bio je na implementaciji algoritma u Python programskom jeziku, gdje su prikazana dva primjera za slučaj modela sa dvije normalne raspodjele. Prvi primjer je preuzet iz [1] gdje su jasno pokazane razlike koje se dobiju u odnosu na autore primjera. Drugi primjer je napravljen od strane autorice teksta i dalje detaljnije obrađuje dobivene rezultate. Na kraju rada je dat prilog sa svim korištenim matematičkim izvođenjima i prilog sa implementacijom algoritma u Python programskom jeziku.

Uvod

Expectation Maximization algoritam (u daljem tekstu EM algoritam) je algoritam koji se koristi za pojednostavljivanje teških problema maksimalne vjerovatnoće¹. Često je potrebno za mnoge modele u statistici i mašinskom učenju izračunati maksimalnu vjerovatnoću ili uraditi procjenu MAP² parametara.

Računanje prethodno navedenih parametara je lagano ukoliko je moguće opservirati sve vrijednosti relevantnih slučajnih varijabli tj. ako radimo sa skupom kompletnih podataka. Međutim, problem nastaje ukoliko imamo podatke koji nedostaju i/ili latentne³ varijable; u tom slučaju računanje ML-a i procjena parametara MAP-a postaje teško.

EM algoritam je jednostavan iterativni algoritam, koji često ima ažuriranja izraza zatvorenog tipa⁴ u svakom koraku. Nadalje, ovaj algoritam automatski prisiljava sva potrebna ograničenja.

Koristi se činjenica da, kada bi podaci bili u potpunosti opservirani, onda bi ML/MAP procjena bila lagana za izračunati. Posebno, EM algoritam je iterativni algoritam koji alternira između procjene parametara koji nedostaju - *Expectation* korak (u daljem tekstu E korak), te optimiziranja (maksimiziranja) parametara kada su dodani parametri koji nedostaju - *Maximization* korak (u daljem tekstu M korak).

U nastavku je prikazan ukratko generalizirani EM algoritam, te nakon toga je rad fokusiran na EM algoritam za model koji sadrži dvije normalne raspodjele - gdje je urađena implementacija algoritma i pokazana na dva primjera.

¹eng. *ML* - *Maximum Likelihood*

²lat. *MAP* - *Maximum a posteriori*

³latentno - prikriveno, skriveno

⁴eng. *closed-form expressions* - izrazi zatvorenog tipa su izrazi koji sadrže konačan broj standardnih operacija (kao što su npr. sabiranje, oduzimanje, množenje i dijeljenje)

Generalizirani EM algoritam

Ovo poglavlje, uključujući sve formule i definicije, je adaptirano iz [2] i [1].

2.1 Osnovna ideja

Neka je $\mathbf{x}_i$ vektor opserviranih varijabli u slučaju i i neka je $\mathbf{z}_i$ vektor skrivenih varijabli. Cilj je maksimizirati tzv. logaritamsku vjerovatnoću (definicija data pod A.3) opserviranih podataka:

$$l(\boldsymbol{\theta}) = \sum_{i=1}^N \log p(\mathbf{x}_i | \boldsymbol{\theta}) = \sum_{i=1}^N \log \left[\sum_{\mathbf{z}_i} p(\mathbf{x}_i, \mathbf{z}_i | \boldsymbol{\theta}) \right] \quad (2.1)$$

Nažalost, jednačinu 2.1 je teško optimizirati, jer logaritam se ne može ubaciti unutar sume. EM algoritam ovo rješava na sljedeći način. Pišemo logaritamsku vjerovatnoću kompletnih podataka kao:

$$l_C(\boldsymbol{\theta}) \triangleq \sum_{i=1}^N \log p(\mathbf{x}_i, \mathbf{z}_i | \boldsymbol{\theta}) \quad (2.2)$$

Jednačina 2.2 se ne može izračunati zbog toga što je $\mathbf{z}_i$ nepoznato. Definirajmo onda očekivanu logaritamsku vjerovatnoću kompletnih podataka kao:

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{t-1}) = \mathbb{E} [l_C(\boldsymbol{\theta}) | \mathcal{D}, \boldsymbol{\theta}^{t-1}] \quad (2.3)$$

gdje je t trenutna iteracija. Q se zove pomoćna funkcija¹. Očekivanje se računa preko parametara iz prethodne iteracije, $\boldsymbol{\theta}^{t-1}$, i opserviranih podataka $\mathcal{D}$. Cilj E koraka je da se izračuna $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{t-1})$ ili, bolje rečeno, parametri unutar $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{t-1})$ od kojih MLE² zavisi. U M koraku (maksimizacijski korak), optimiziramo Q funkciju u zavisnosti od $\boldsymbol{\theta}$:

$$\boldsymbol{\theta}^t = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{t-1}) \quad (2.4)$$

Da bi se uradila MAP estimacija, M korak se modificira na sljedeći način:

$$\boldsymbol{\theta}^t = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{t-1}) + \log p(\boldsymbol{\theta}) \quad (2.5)$$

E korak ostaje nepromijenjen.

Važno je naglasiti da EM algoritam monotono povećava logaritamsku vjerovatnoću posmatranih podataka ili on ostaje isti kako se povećava broj iteracija.[2] Logaritamsku vjerovatnoću posmatranih podataka koristimo kao kriterij konvergencije u tom slučaju, kada logaritamska vjerovatnoća postane približno jednaka u trenutnoj iteraciji u odnosu

¹eng. auxiliary function

²eng. MLE - *Maximum Likelihood Estimation* tj. procjena maksimalne vjerovatnoće

na prethodnu, tada je algoritam konvergirao.[2] [1] U narednom poglavlju ćemo još proširiti ovu tvrdnju.

2.2 EM za GMM

Prvo definiramo GMM - *Gaussian Mixture Models*³ što su zapravo modeli koji sadrže mješavinu više normalnih raspodjela. Postoje modeli koji sadrže i druge raspodjele, kao što i postoji onda odgovarajuća modifikacija EM algoritma za te raspodjele; [2] ali ovaj rad se fokusira samo na EM algoritam za GMM.

Pretpostavljamo da je broj miješanih komponenti jednak K i da je on poznat. Pomoćna funkcija (preuzimamo samo krajnju vrijednost formule iz [2], bez izvođenja) tada postaje:

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t-1)}) \triangleq \mathbb{E} \left[\sum_i \log p(\mathbf{x}_i, z_i | \boldsymbol{\theta}) \right] \quad (2.6)$$

$$= \sum_i \sum_k r_{ik} \log \pi_k + \sum_i \sum_k r_{ik} \log p(\mathbf{x}_i | \boldsymbol{\theta}_k) \quad (2.7)$$

gdje je $r_{ik} \triangleq p(z_i = k | \mathbf{x}_i, \boldsymbol{\theta}^{(t-1)})$ *responsibility*⁴ koju klaster k uzima za podatkovnu tačku i . Ono se računa u E koraku:

$$r_{ik} = \frac{\pi_k p(\mathbf{x}_i | \boldsymbol{\theta}_k^{(t-1)})}{\sum_{k'} \pi_{k'} p(\mathbf{x}_i | \boldsymbol{\theta}_{k'}^{(t-1)})} \quad (2.8)$$

U M koraku optimiziramo Q po $\boldsymbol{\pi}$ i $\boldsymbol{\theta}_k$. Za $\boldsymbol{\pi}$ imamo:

$$\pi_k = \frac{1}{N} \sum_i r_{ik} = \frac{r_k}{N} \quad (2.9)$$

gdje je $r_k \triangleq \sum_i r_{ik}$ težinski broj tačaka koje su dodijeljene klasteru k . Novi estimatorski parametri su dati sa:

$$\boldsymbol{\mu}_k = \frac{\sum_i r_{ik} \mathbf{x}_i}{r_k} \quad (2.10)$$

$$\sum_k = \frac{\sum_i r_{ik} \mathbf{x}_i \mathbf{x}_i^T}{r_k} - \boldsymbol{\mu}_k \boldsymbol{\mu}_k^T \quad (2.11)$$

Gornje jednačine imaju smisla ako očekivana vrijednost klastera k je samo težinska srednja vrijednost svih tačaka koje su dodijeljene klasteru k , a kovarijansa je proporcionalna težinskoj empirijskoj *scatter*⁵ matrici.

Nakon što se novi parametri izračunaju, onda postavljamo $\boldsymbol{\theta}^t = (\pi_k, \boldsymbol{\mu}_k, \sum_k)$ za $k = 1 : K$, i idemo na sljedeći E korak.

³eng. *Gaussian Mixture Models* - modeli sa mješavinom Gausovih raspodjela

⁴eng. *responsibility* - odgovornost

⁵eng. *scatter matrix* - matrica koja služi da bi se procijenila kovarijantna matrica za npr. multivariabilne normalne raspodjele

Model sa dvije normalne raspodjele

U ovom poglavlju opisujemo EM algoritam za set podataka za koji znamo da se sastoji od dvije normalne raspodjele. Pokazat ćemo način dobivanja parametara obje normalne raspodjele (očekivane vrijednosti i varijanse) kroz pseudokod. Nakon toga, u poglavljima 4 i 5, ćemo pokazati dobivene rezultate za dva primjera - prvi primjer je preuzet iz [1] i sadrži samo 20 podataka u nizu koje treba provući kroz algoritam, dok je drugi primjer napravila autorica ovog teksta kako bi se algoritam demonstrirao na mnogo većem broju podataka u nizu.

3.1 Teorijska pozadina

Sve jednačine u ovom odjeljku su preuzete iz [1].

U ovom pretpostavljenom slučaju znamo da se podaci u našem posmatranom nizu sastoje od podataka koji su sastavljeni iz dvije normalne raspodjele (ili bolje reći možemo uočiti bimodalnosti podataka ukoliko posmatramo njihov histogram). U tom slučaju modeliramo podatke Y kao mješavinu dvije normalne distribucije:

$$Y_1 \sim N(\mu_1, \sigma_1^2) \quad (3.1)$$

$$Y_2 \sim N(\mu_2, \sigma_2^2) \quad (3.2)$$

$$Y = (1 - \Delta) \cdot Y_1 + \Delta \cdot Y_2 \quad (3.3)$$

gdje je $\Delta \in \{0, 1\}$ gdje je $vjer(\Delta = 1) = \pi$. Ova generativna reprezentacija je eksplicitna: treba generisati $\Delta \in \{0, 1\}$, i onda u zavisnosti od rezultata, upisati u Y ili Y_1 (ukoliko se dobije da je $\Delta = 0$) ili Y_2 (ukoliko se dobije da je $\Delta = 1$). Neka je $\phi_\theta(x)$ funkcija gustine vjerovatnoće za normalnu raspodjelu sa parametrima $\theta = (\mu, \sigma^2)$. Onda je gustina vjerovatnoće od Y jednaka:

$$g_Y(y) = (1 - \pi)\phi_{\theta_1}(y) + \pi\phi_{\theta_2}(y) \quad (3.4)$$

Pretpostavimo da želimo da uklopimo ovakav model u naš niz podataka preko maksimalne vjerovatnoće. Onda ti parametri iznose:

$$\theta = (\pi, \theta_1, \theta_2) = (\pi, \mu_1, \mu_2, \sigma_1^2, \sigma_2^2) \quad (3.5)$$

Logaritamska vjerovatnoća za N podataka u nizu iznosi:

$$l(\theta; \mathbf{Z}) = \sum_{i=1}^N \log [(1 - \pi)\phi_{\theta_1}(y_i) + \pi\phi_{\theta_2}(y_i)] \quad (3.6)$$

Kao što je spomenuto u poglavlju 2, direktna maksimizacija $l(\theta; \mathbf{Z})$ je numerički veoma teška, zbog sume argumenata unutar logaritma. Ipak, postoji jednostavniji pristup!

Uzmimo u obzir neopservirane latentne varijable Δ_i koje uzimaju vrijednosti 0 ili 1 kao u 3.3: ako je $\Delta_i = 1$ onda Y_i je iz modela 2, a u suprotnom je iz modela 1. Pretpostavimo da znamo vrijednosti Δ_i . Onda je logaritamska vjerovatnoća:

$$l_0(\theta; \mathbf{Z}, \mathbf{\Delta}) = \sum_{i=1}^N [(1 - \Delta_i) \log \phi_{\theta_1}(y_i) + \Delta_i \log \phi_{\theta_2}(y_i)] \quad (3.7)$$

$$+ \sum_{i=1}^N [(1 - \Delta_i) \log(1 - \pi) + \Delta_i \log \pi] \quad (3.8)$$

i procjene maksimalne vjerovatnoće od μ_1 i σ_1^2 su očekivana vrijednost i varijansa datih podataka gdje je $\Delta_i = 0$, a slično za μ_2 i σ_2^2 su očekivana vrijednost i varijansa datih podataka za $\Delta_i = 1$.¹

Kako su vrijednosti Δ_i zapravo nepoznate, algoritam se izvršava iterativno, gdje za svako Δ_i mijenjamo njegovu očekivanu vrijednost koja je data izrazom:

$$\gamma_i(\theta) = E(\Delta_i | \theta, \mathbf{Z}) = vjer(\Delta_i = 1 | \theta, \mathbf{Z}) \quad (3.9)$$

Izraz 3.9 se isto tako zove *odgovornost*² modela 2 za opservaciju i .

Dakle, u EM algoritmu, u E koraku uradimo dodjelu svake opservacije svakom modelu: trenutna procjena parametara se koristi da se dodijele odgovornosti prema relativnoj gustini tačaka u svakom modelu, dok u M koraku, ove odgovornosti se koriste u računanju težinskih koeficijenata maksimalne vjerovatnoće da bi se ažurirale procjene parametara.

Inicijalizacija početnih $\hat{\mu}_1$ i $\hat{\mu}_2$ se može uraditi tako što im se samo dodijele različite vrijednosti nekih od tačaka y_i iz datih podataka nasumično. $\hat{\sigma}_1^2$ i $\hat{\sigma}_2^2$ mogu se na početku inicijalizirati na vrijednost varijanse datih tačaka: $\sum_{i=1}^N (y_i - \bar{y})^2 / N$. Miješana proporcija $\hat{\pi}$ se može postaviti na početku na vrijednost 0.5.

Trebamo primijetiti da stvarna maksimizacija vjerovatnoće se javlja onda kada stavimo vrijednost beskonačne visine kao jednu tačku, tj. $\hat{\mu}_1 = y_i$ za neko i i $\hat{\sigma}_1^2 = 0$. Ovo daje beskonačnu vjerovatnoću, ali nije dobro rješenje. Dakle, zapravo tražimo dobar lokalni maksimum vjerovatnoće, za koji vrijedi da je $\hat{\sigma}_1^2, \hat{\sigma}_2^2 > 0$. Da se stvari još zakompliciraju, može biti više od jednog lokalnog maksimuma za koji vrijedi $\hat{\sigma}_1^2, \hat{\sigma}_2^2 > 0$. Zbog toga se koristi kao kriterij konvergencije logaritamska vjerovatnoća, jer uz pomoć nje jasno vidimo kada se vrijednosti estimatora prestanu mijenjati u odgovarajućoj okolini koju mi zadamo unutar algoritma.

3.2 Pseudokod za EM algoritam za model sa dvije normalne raspodjele

1 Inicijalizirati parametre $\hat{\mu}_1, \hat{\mu}_2, \hat{\sigma}_1^2, \hat{\sigma}_2^2, \hat{\pi}$ (objašnjeno u tekstu)

2 *Expectation* korak: izračunati odgovornosti

$$\hat{\gamma}_i = \frac{\hat{\pi} \phi_{\hat{\theta}_2}(y_i)}{(1 - \hat{\pi}) \phi_{\hat{\theta}_1}(y_i) + \hat{\pi} \phi_{\hat{\theta}_2}(y_i)}, \quad i = 1, 2, \dots, N \quad (3.10)$$

3 *Maximization* korak: izračunati težinske očekivane vrijednosti i varijanse:

$$\hat{\mu}_1 = \frac{\sum_{i=1}^N (1 - \hat{\gamma}_i) y_i}{\sum_{i=1}^N (1 - \hat{\gamma}_i)} \quad (3.11)$$

¹Pogledati A za detaljnije objašnjenje logaritamske vjerovatnoće i pogledati [3] za jedan od načina kako se koristi za ograničavanje logaritma sume odozdo

²eng. *responsibility*

$$\hat{\mu}_2 = \frac{\sum_{i=1}^N \hat{\gamma}_i y_i}{\sum_{i=1}^N \hat{\gamma}_i} \quad (3.12)$$

$$\hat{\sigma}_1^2 = \frac{\sum_{i=1}^N (1 - \hat{\gamma}_i)(y_i - \hat{\mu}_1)^2}{\sum_{i=1}^N (1 - \hat{\gamma}_i)} \quad (3.13)$$

$$\hat{\sigma}_2^2 = \frac{\sum_{i=1}^N \hat{\gamma}_i (y_i - \hat{\mu}_2)^2}{\sum_{i=1}^N \hat{\gamma}_i} \quad (3.14)$$

$$\hat{\pi} = \sum_{i=1}^N \frac{\hat{\gamma}_i}{N} \quad (3.15)$$

4 Iterirati korake 2 i 3 dok algoritam ne konvergira.

Primjer 1

Ovaj primjer je preuzet iz [1]. Neka je dato Y koje sadrži 20 tačaka koje su predstavljene u tabeli 4.1. Koristeći algoritam koji je dat u prethodnom dijelu ovog teksta, koji je implementiran onako kako je pokazano u B, dobijaju se krajnje vrijednosti za parametre: $\hat{\mu}_1 = 4.65494$, $\hat{\mu}_2 = 1.08232$, $\hat{\sigma}_1^2 = 0.82022$, $\hat{\sigma}_2^2 = 0.81001$ i $\hat{\pi} = 0.55434$. Algoritam konvergira nakon $k = 38$ koraka.

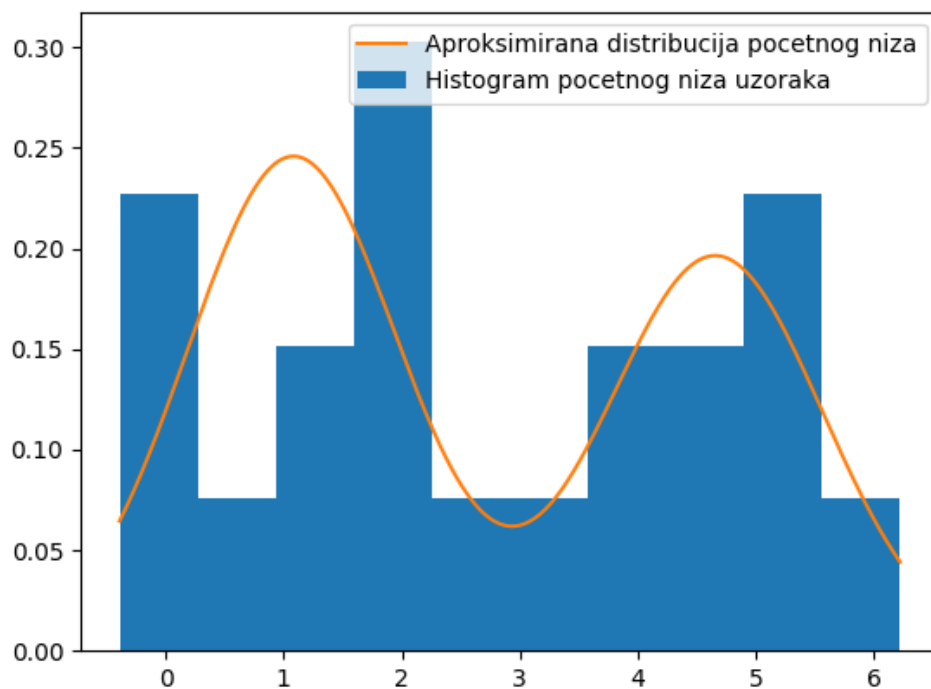
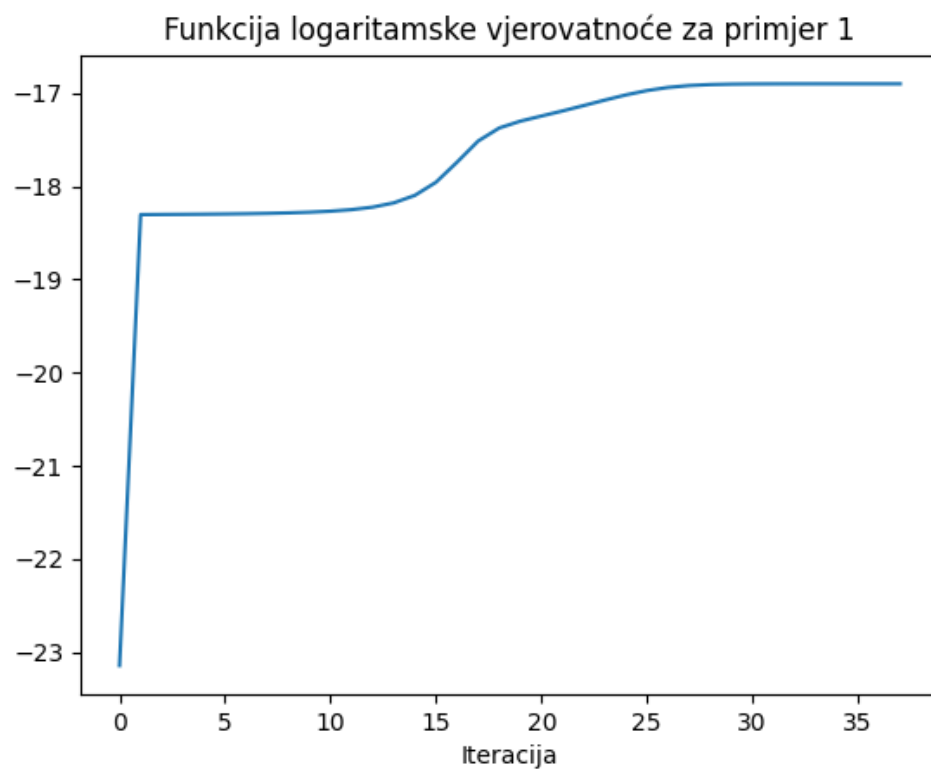
-0.39	0.12	0.94	1.67	1.76	2.44	3.72	4.28	4.92	5.53
0.06	0.48	1.01	1.68	1.80	3.25	4.12	4.60	5.28	6.22

Tablica 4.1: Podaci koji sačinjavaju Y u primjeru 1

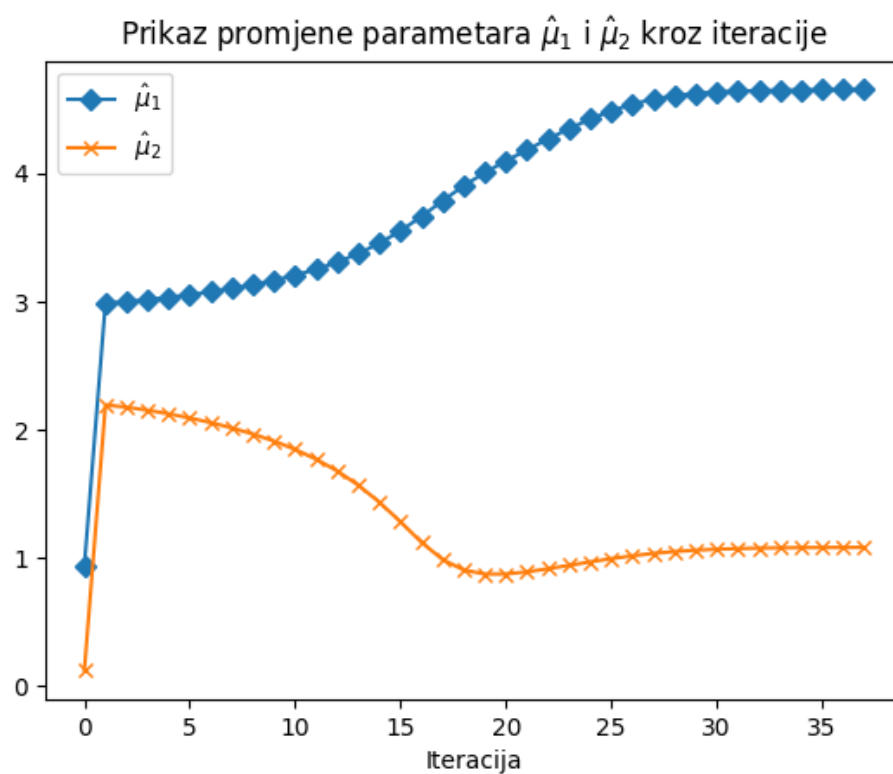
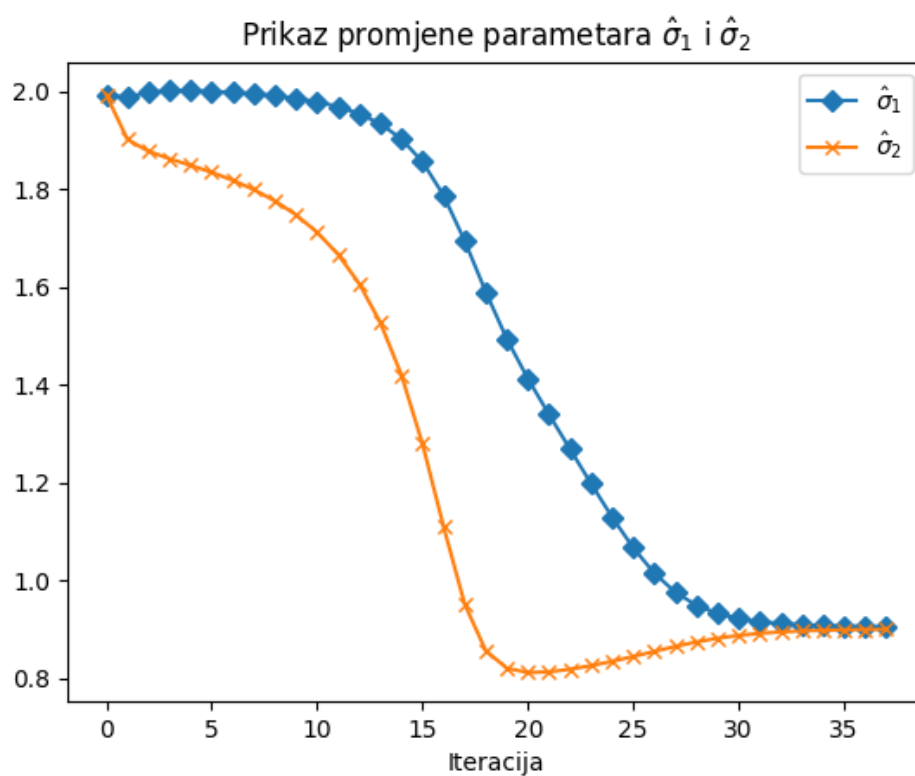
Na slici 4.1 je prikazan histogram datih vrijednosti Y (plava boja) kao i aproksimacija zadanih tačaka sa dvije normalne raspodjele (narandžasta boja). Na tom grafiku možemo vidjeti kako je aproksimacija približno dobra za ovakav niz brojeva, ali tek se u narednom primjeru može vidjeti zapravo koliko dobro EM algoritam radi, kada budemo imali Y sa mnogo više tačaka.

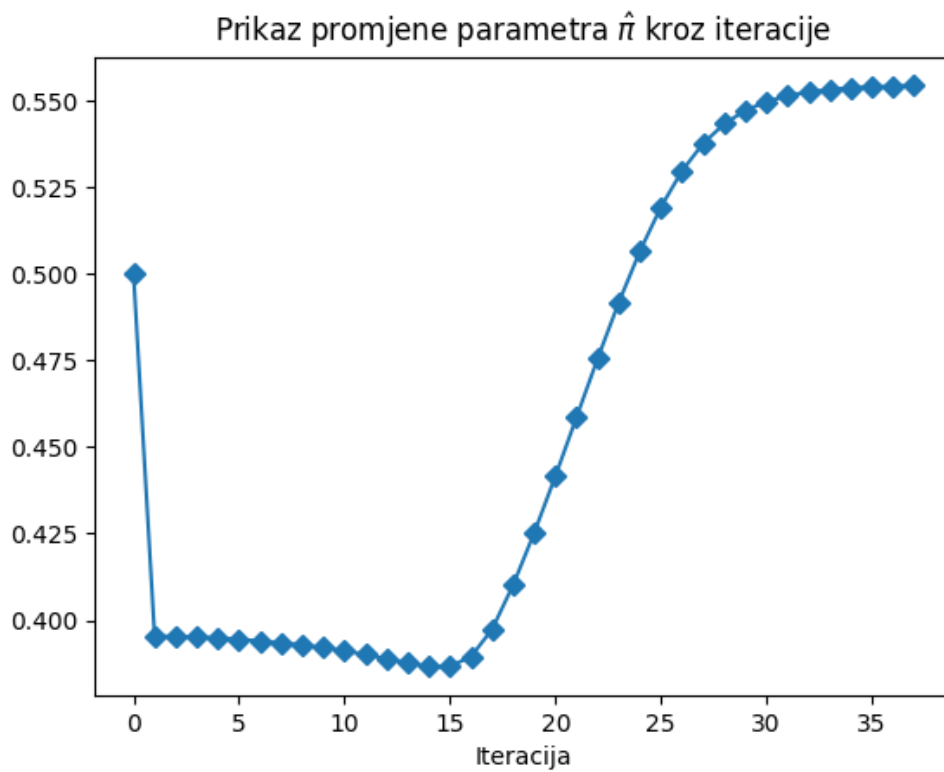
Na slici 4.2 je prikazana logaritamska vjerovatnoća za primjer 1, gdje možemo uočiti da za 36. i 37. iteraciju (iteracije počinju od broja 0), logaritamska vjerovatnoća se ne razlikuje mnogo (kao što se može vidjeti u priloženom kodu u B postavljen je kriterij konvergencije na 10^{-10}), čime je algoritam konvergirao.

Na slikama 4.3, 4.4 i 4.5 su prikazane promjene parametara $\hat{\mu}_1$, $\hat{\mu}_2$, $\hat{\sigma}_1^2$, $\hat{\sigma}_2^2$ i $\hat{\pi}$ kroz sve iteracije. Možemo vidjeti da slično kao i funkcija logaritamske vjerovatnoće, vrijednosti parametara u zadnjih 8 iteracija su približno jednake i približavaju se krajnjoj vrijednosti koju dobijemo kada algoritam konačno konvergira.

Slika 4.1: Histogram Y i dobivena aproksimacija za Y u primjeru 1

Slika 4.2: Logaritamska vjerovatnoća za primjer 1

Slika 4.3: Grafik koji prikazuje promjene parametara $\hat{\mu}_1$ i $\hat{\mu}_2$ kroz iteracije za primjer 1Slika 4.4: Grafik koji prikazuje promjene parametara $\hat{\sigma}_1$ i $\hat{\sigma}_2$ kroz iteracije za primjer 1



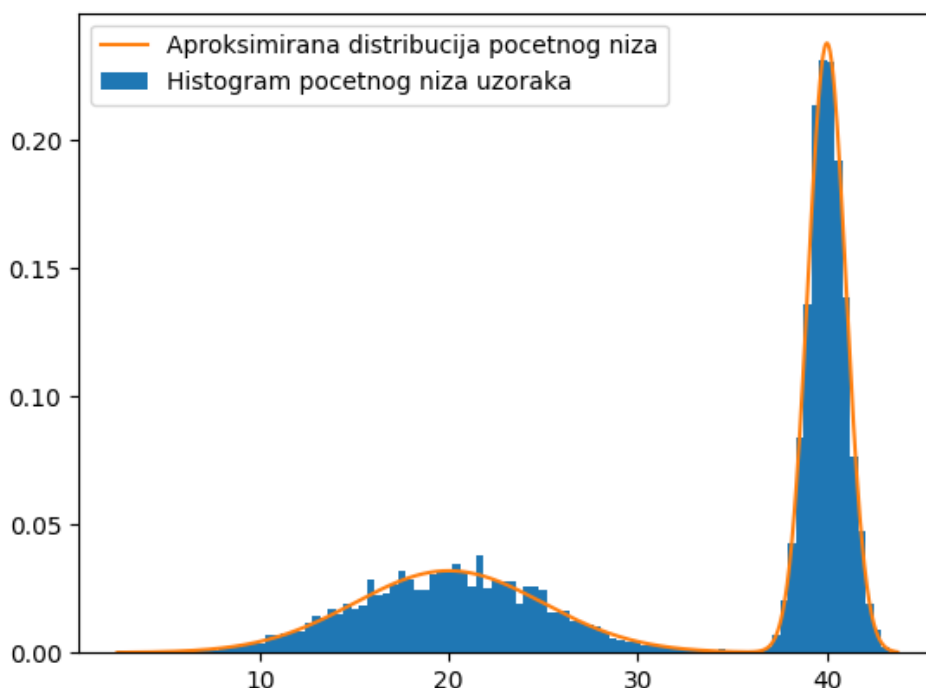
Slika 4.5: Grafik koji prikazuje promjenu parametra $\hat{\pi}$ kroz iteracije za primjer 1

U knjizi [1] autori su dobili vrijednost parametara: $\hat{\mu}_1 = 4.62$, $\hat{\mu}_2 = 1.06$, $\hat{\sigma}_1^2 = 0.87$, $\hat{\sigma}_2^2 = 0.77$ i $\hat{\pi} = 0.546$, i njihov algoritam je konvergirao za $k = 20$ iteracija. Razlike u krajnjim parametrima dolaze od biranja početnih vrijednosti za $\hat{\mu}_1$ i $\hat{\mu}_2$, te se nisu mogli dobiti identični rezultati kao kod autora jer oni nisu otkrili koje su početne vrijednosti izabrali, nego su samo dali krajnji rezultat za tražene parametre. Svakako su dobiveni parametri iznimno dobri jer se razlikuju u drugoj decimali od parametara koje su autori dobili u knjizi.

Primjer 2

Primjer 2 je autorica ovog teksta sama napravila tako što je napravila niz koji se sastoji od dvije gausove raspodjele sa parametrima: $\mu_1 = 20$, $\mu_2 = 40$, $\sigma_1 = 5$ i $\sigma_2 = 1$. Prva normalna raspodjela sadrži 4000 tačaka, dok druga raspodjela sadrži 6000 tačaka. Kao što se može vidjeti u B, navedene dvije raspodjele su stavljene u jedan niz (y) i onda se taj niz promiješa kako bi se dobile nasumične vrijednosti za početno $\hat{\mu}_1$ i $\hat{\mu}_2$ koji su stavljene da su jednaki prvom i drugom članu tako promiješanog niza.

Za $\hat{\sigma}_1$ i $\hat{\sigma}_2$ su uzete vrijednosti kao korijen iz varijanse niza tačaka y , dok je za $\hat{\pi}$ početnu vrijednost uzeto 0.5. Nakon provođenja algoritma, dobijaju se vrijednosti parametara: $\hat{\mu}_1 = 40.01109$, $\hat{\mu}_2 = 19.99689$, $\hat{\sigma}_1^2 = 1.01086$, $\hat{\sigma}_2^2 = 24.94299$ i $\hat{\pi} = 0.40012$ i algoritam konvergira nakon $k = 19$ iteracija.



Slika 5.1: Histogram Y i dobivena aproksimacija za Y u primjeru 2

μ_1 je vezano za desnu, dok je μ_2 vezano za lijevu normalnu raspodjelu, tako da vidimo da se dobiju rezultati jednaki početnim postavljenim vrijednostima sa kojima se zapravo generisao niz y . To dokazuje da algoritam radi dobro, i možemo sada na grafiku na slici 5.1 vidjeti tačno koliko se dobro sada aproksimirana distribucija podudara sa histogramom

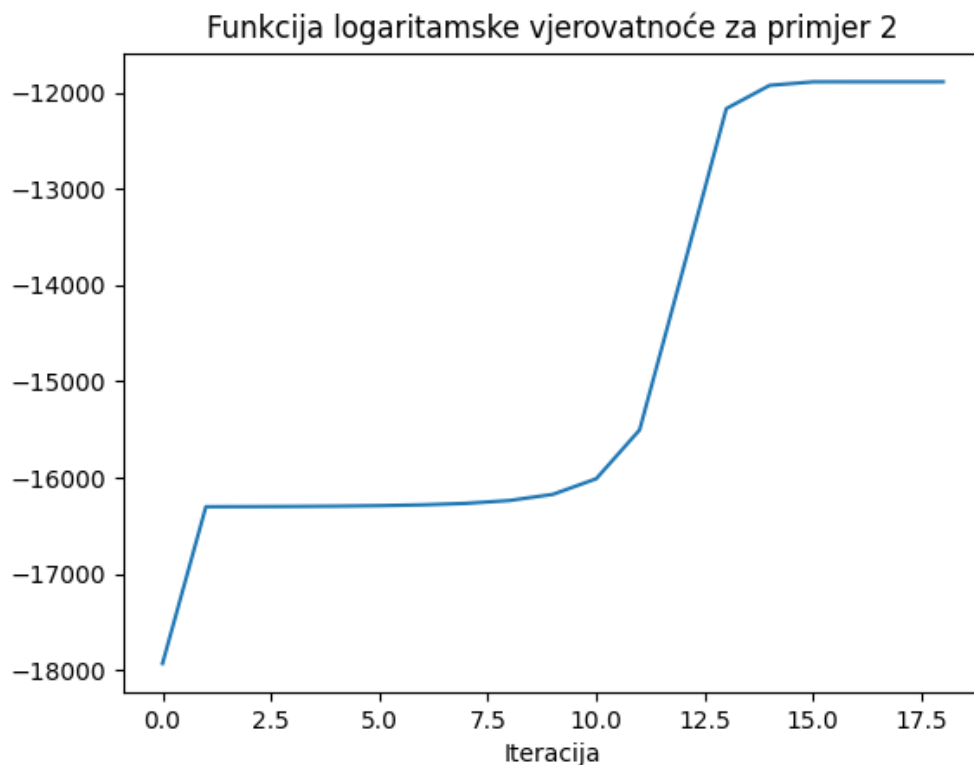
početnog niza uzoraka. Vidimo da kada imamo niz sa puno više tačaka, da se bolji rezultat dobije u odnosu na primjer 1.

Na slici 5.2 vidimo logaritamsku vjerovatnoću za primjer 2. Kada bismo uvećali zadnje 3 iteracije, vidjeli bismo da tek u 17. i 18. iteraciji (iteracije počinju od broja 0) se uslov konvergencije ispunio (postavljeno je da razlika logaritamska vjerovatnoća treba biti manja od 10^{-6}). Iako na grafiku izgleda da su prilično blizu i 16. i 17. iteracija, ipak se trebamo sjetiti da je ovdje u pitanju logaritamska funkcija tako da se ne mogu vrijednosti tek tako procijeniti golim okom (što se može lako provjeriti ako provjerimo vrijednosti niza za logaritamsku vjerovatnoću i kada uzmemo zadnje 4 vrijednosti u tom nizu - u kodu nazvan "LL").

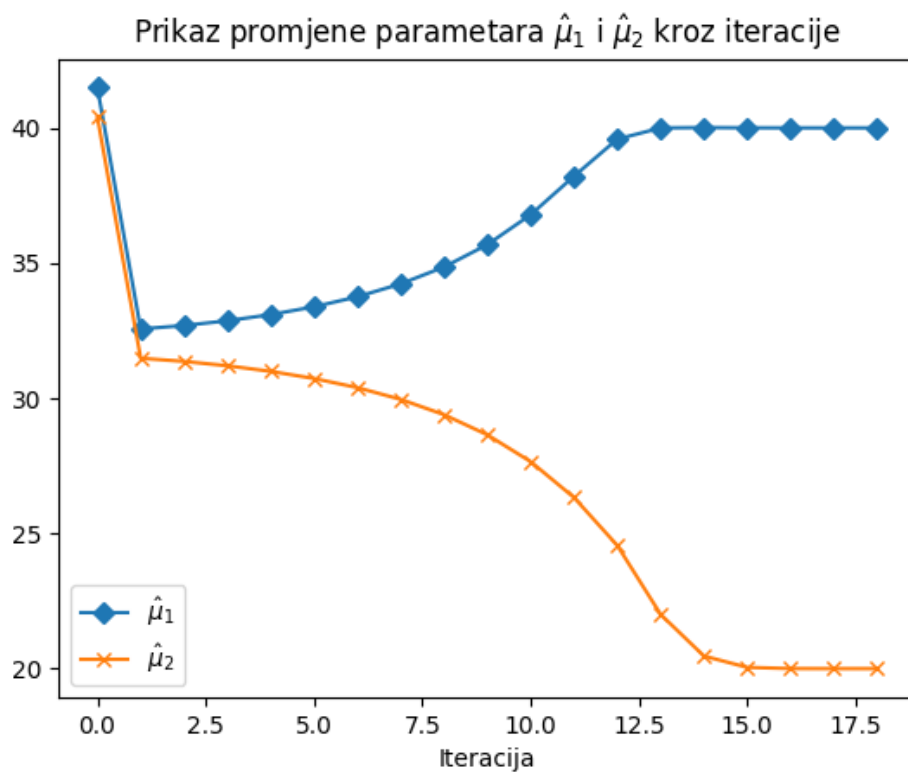
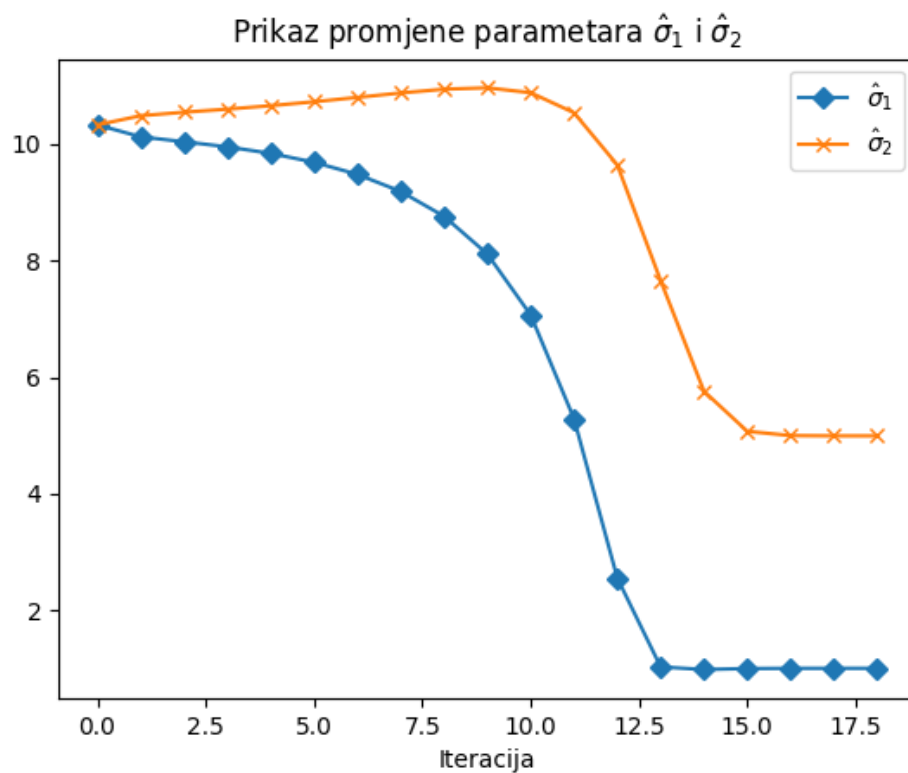
Kao za primjer 1, na slikama 5.3, 5.4 i 5.5 smo prikazali promjene parametara $\hat{\mu}_1$, $\hat{\mu}_2$, $\hat{\sigma}_1^2$, $\hat{\sigma}_2^2$ i $\hat{\pi}$ kroz sve iteracije. Možemo uočiti da su u zadnje 4 iteracije odgovarajuće vrijednosti ovih parametara približno jednaki i prilaze konačnim svojim vrijednostima gdje na kraju algoritam konvergira.

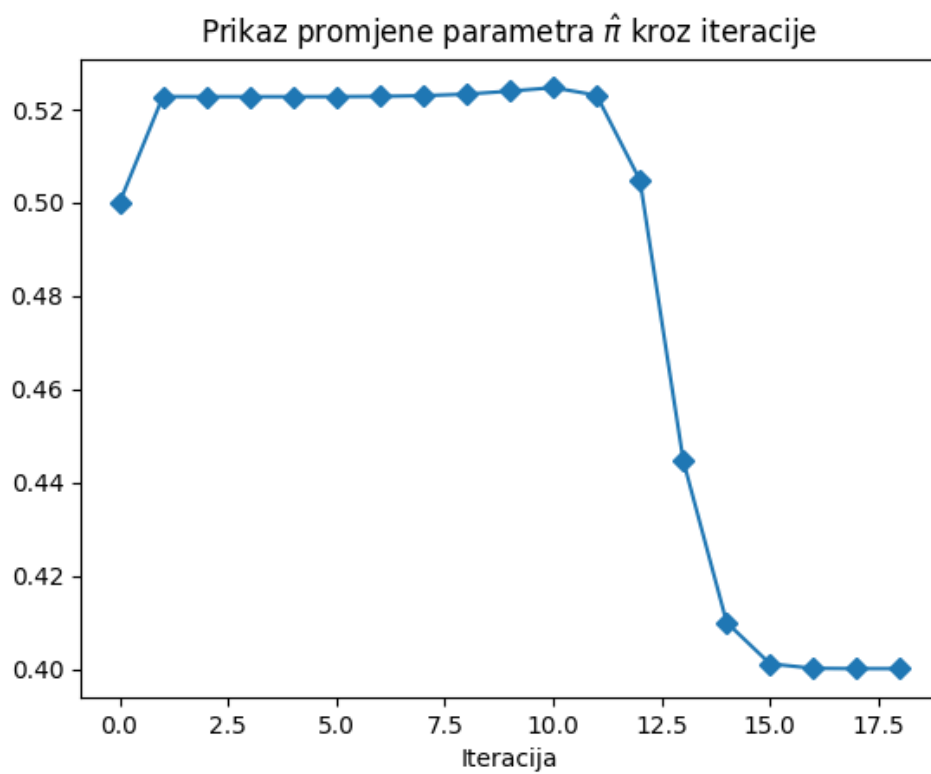
Vrijednost parametra $\hat{\pi}$ pokazuje da je vjerovatnoća da je slučajna varijabla iz normalne raspodjele koja ima $\mu = 20$ približno jednaka 0.4 - što je i za očekivati jer tačaka iz te raspodjele ima 4000, dok tačaka iz raspodjele gdje je $\mu = 40$ ima 6000.

Bitno je naglasiti da svakim novim pokretanjem simulacije će se dobiti drugačiji izračunati parametri (razlike u parametrima će biti minorne, ali opet ih treba istaći), što je i očekivano jer svaki put uzimamo drugačije vrijednosti za početne vrijednosti zbog toga što se niz y promiješa. Pored toga, svaki put kada se simulacija nanovo pokrene, dobijamo različito generisane podatke za dvije normalne raspodjele koje sačinjavaju niz y .



Slika 5.2: Logaritamska vjerovatnoća za primjer 2

Slika 5.3: Grafik koji prikazuje promjene parametara $\hat{\mu}_1$ i $\hat{\mu}_2$ kroz iteracije za primjer 2Slika 5.4: Grafik koji prikazuje promjene parametara $\hat{\sigma}_1$ i $\hat{\sigma}_2$ kroz iteracije za primjer 2



Slika 5.5: Grafik koji prikazuje promjenu parametra $\hat{\pi}$ kroz iteracije za primjer 2

Zaključak

Vidimo da je EM algoritam veoma moćan alat pri računanju ML-a i procjeni MAP parametara. U ovom radu je fokus bio na prezentiranju jedne jednostavne solucije koja je tretirala model sa dvije nepoznate normalne raspodjele. Jasno je da se algoritam može proširiti na model koji sadrži k normalnih raspodjela, onako kako je pokazano u [2], kao i da se može proširiti na model koji sadrži n neke druge raspodjele. Pored toga, postoji mnogo verzija EM algoritma koje su prikazane u [2], a koje su isuviše kompleksne da bi se izučavale u ovom radu.

Dva bitna zaključka koja možemo izvesti za EM algoritam koji je implementiran kako je prikazano u B za model sa dvije normalne raspodjele su: broj iteracija dok algoritam ne konvergira zavisi od izbora početnih parametara i bolja procjena parametara se obavlja što je veći broj podataka u nizu podataka u kojem tražimo navedene parametre.

Autorica teksta se nada da će ovaj rad biti dobra polazna tačka za zainteresovane koji se prvi put susreću sa ovakvim problemom i da će navedeni izvori informacija koristiti drugima za dalje, i potencijalno dublje, istraživanje.

Korištene definicije i teoreme

Teorema A.1 (Rolleov teorem). [4] Ako je f takva funkcija da:

- i) f je kontinualna na zatvorenom intervalu $[a, b]$,
- ii) $f'(x)$ postoji za svaku tačku x na otvorenom intervalu (a, b)
- iii) $f(a) = f(b)$ (ili kao specijalan slučaj $f(a) = f(b) = 0$),

postoji tačka c gdje je $a < c < b$ tako da je $f'(c) = 0$.

Dokaz. Kako je $f(a) = f(b)$, osim ako je funkcija f konstantna pa u tom slučaju je teorema odmah dokazana, f treba da ili raste ili opada kada x uzima vrijednost veću od a . Pretpostavimo da raste, onda pošto uzima vrijednost $f(b) = f(a)$, mora prestati da raste i početi da opada u nekoj tački c tako da je $a < c < b$. U toj tački, pošto postoji maksimalna vrijednost funkcije, onda vrijedi da su $f(c+h) - f(c)$ i $f(c-h) - f(c)$ negativni (gdje je h pozitivno ali malo).

$$\frac{f(c+h) - f(c)}{h} < 0 \wedge \frac{f(c-h) - f(c)}{-h} > 0$$

Dakle, kako h teži ka nuli gornji izrazi teže da budu pozitivni i negativni, respektivno, osim ako oba izraza imaju limes koji je jednak nuli. Ukoliko imaju različite limese, onda:

$$Rf'(c) \neq Lf'(c)$$

i u tom slučaju $f'(c)$ ne postoji, što je u kontradikciji sa hipotezom teoreme. Dakle, oba limesa moraju biti jednaki nuli ili:

$$f'(c) = 0 \text{ za } a < c < b$$

QED

Teorema A.2 (Lagrangeov teorem srednje vrijednosti). [4] Ako je funkcija f :

- i) kontinualna na zatvorenom intervalu $[a, b]$ i
- ii) diferencijabilna na otvorenom intervalu (a, b) ,

onda postoji makar jedna tačka c na otvorenom intervalu (a, b) , takva da je:

$$\frac{f(b) - f(a)}{b - a} = f'(c)$$

Dokaz. Razmotrimo funkciju Φ :

$$\Phi(x) = f(x) + Ax$$

gdje je A konstanta koja se treba odrediti, tako da vrijedi $\Phi(a) = \Phi(b)$. To nam daje:

$$f(a) + Aa = f(b) + Ab$$

ili:

$$-A = \frac{f(b) - f(a)}{b - a}$$

Kako je dato da je f diferencijabilna na (a, b) i x je diferencijabilna funkcija na bilo kojem intervalu, slijedi da je $\Phi(x) = f(x) + Ax$ je diferencijabilna na (a, b) . Dakle, svi uvjeti teoreme A.1 su zadovoljeni za funkciju Φ . Onda postoji makar jedna tačka c na otvorenom intervalu (a, b) tako da vrijedi $\Phi'(c) = 0$. Ali vrijedi:

$$\Phi'(x) = f'(x) + A$$

$$0 = \Phi'(c) = f'(c) + A \quad \vee \quad -A = f'(c)$$

Iz gore istaknutih jednačina onda možemo izraziti:

$$\frac{f(b) - f(a)}{b - a} = f'(c)$$

QED

Definicija A.1 (Konveksna funkcija). [3] Neka je f realna funkcija definirana na intervalu $I = [a, b]$. Za f se kaže da je konveksna na I ukoliko $\forall x_1, x_2 \in I, \lambda \in [0, 1]$ vrijedi:

$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$$

f je striktno konveksna ukoliko je gornja nejednakost strogo manja za sve različite x_1 i x_2 , te ukoliko $\lambda \in (0, 1)$.

Definicija A.2 (Konkavna funkcija). [3] f je konkavna (strogo konkavna) ukoliko je $-f$ konveksna (strogo konveksna).

Teorema A.3. [3] Ako je $f(x)$ dvostruko diferencijabilna na $[a, b]$ i $f''(x) \geq 0$ na $[a, b]$, onda je $f(x)$ konveksna na $[a, b]$.

Dokaz. Za $x \leq y \in [a, b]$ i $\lambda \in [0, 1]$ neka je $z = \lambda y + (1 - \lambda)x$. Prema definiciji A.1 f je konveksna akko $f(\lambda y + (1 - \lambda)x) \leq \lambda f(y) + (1 - \lambda)f(x)$. Ako napišemo $z = \lambda y + (1 - \lambda)x$ i uočimo da je $f(z) = \lambda f(z) + (1 - \lambda)f(z)$, onda imamo da je $f(z) = \lambda f(z) + (1 - \lambda)f(x) \leq \lambda f(y) + (1 - \lambda)f(x)$. Ako ispremještamo izraze, dobijamo ekvivalentnu definiciju konveksnosti: f je konveksna ako

$$\lambda[f(y) - f(z)] \geq (1 - \lambda)[f(z) - f(x)]$$

Prema Lagrangeovoj teoremi A.2, $\exists s, x \leq s \leq z$, tako da:

$$f(z) - f(x) = f'(s)(z - x)$$

Slično, ako teoremu A.2 primjenimo na $f(y) - f(z), \exists t, z \leq t \leq y$, tako da:

$$f(y) - f(z) = f'(t)(y - z)$$

Tako da imamo: $x \leq s \leq z \leq t \leq y$. Prema pretpostavci, $f''(x) \geq 0$ na $[a, b]$ tako da:

$$f'(s) \leq f'(t)$$

kako je $s \leq t$. Možemo napisati $z = \lambda y + (1 - \lambda)x$ kao:

$$(1 - \lambda)(z - x) = \lambda(y - z)$$

Konačno, kombinacijom dobivenih izraza dobijamo:

$$\begin{aligned} (1 - \lambda)[f(z) - f(x)] &= (1 - \lambda)f'(s)(z - x) \\ &\leq f'(t)(1 - \lambda)(z - x) \\ &= \lambda f'(t)(y - z) \\ &= \lambda[f(y) - f(z)] \end{aligned}$$

QED

Tvrdnja A.1. [3] $\ln(x)$ je striktno konkavna na $(0, \infty)$.

Dokaz. Kako je $f(x) = -\ln(x)$, onda imamo da je $f''(x) = \frac{1}{x^2} > 0$ za $(x \in (0, \infty))$. Prema teoremi A.3 $-\ln(x)$ je striktno konvexna na $(0, \infty)$. Prema definiciji A.2 $\ln(x)$ je striktno konkavna na $(0, \infty)$. QED

Teorema A.4 (Jensenova nejednakost). [3] Neka je f konvexna funkcija definirana na intervalu I . Ako su $x_1, x_2, \dots, x_n \in I$ i $\lambda_1, \lambda_2, \dots, \lambda_n \geq 0$ gdje $\sum_{i=1}^n \lambda_i = 1$, onda vrijedi:

$$f\left(\sum_{i=1}^n \lambda_i x_i\right) \leq \sum_{i=1}^n \lambda_i f(x_i)$$

Dokaz. Za $n = 1$ dokaz je trivijalan. Za $n = 2$ teorema se svodi na definiciju konveksnosti funkcije koja je data pod definicijom A.1. Da bi dokazali da teorema vrijedi za sve prirodne brojeve, koristit ćemo matematičku indukciju. Pretpostavimo da je teorema tačna za neko n , onda vrijedi:

$$\begin{aligned} f\left(\sum_{i=1}^{n+1} \lambda_i x_i\right) &= f\left(\lambda_{n+1} x_{n+1} + \sum_{i=1}^n \lambda_i x_i\right) \\ &= f\left(\lambda_{n+1} x_{n+1} + (1 - \lambda_{n+1}) \frac{1}{(1 - \lambda_{n+1})} \sum_{i=1}^n \lambda_i x_i\right) \\ &\leq \lambda_{n+1} f(x_{n+1}) + (1 - \lambda_{n+1}) f\left(\frac{1}{1 - \lambda_{n+1}} \sum_{i=1}^n \lambda_i x_i\right) \\ &= \lambda_{n+1} f(x_{n+1}) + (1 - \lambda_{n+1}) f\left(\sum_{i=1}^n \frac{\lambda_i}{1 - \lambda_{n+1}} x_i\right) \\ &\leq \lambda_{n+1} f(x_{n+1}) + (1 - \lambda_{n+1}) \sum_{i=1}^n \frac{\lambda_i}{1 - \lambda_{n+1}} f(x_i) \\ &= \lambda_{n+1} f(x_{n+1}) + \sum_{i=1}^n \lambda_i f(x_i) \\ &= \sum_{i=1}^{n+1} \lambda_i f(x_i) \end{aligned}$$

QED

Tvrđnja A.2. [3] *Kako je $\ln(x)$ konkavna na osnovu definicije A.2, možemo iskoristiti Jensenovu nejednakost da bismo dobili rezultat:*

$$\ln \sum_{i=1}^n \lambda_i x_i \geq \sum_{i=1}^n \lambda_i \ln(x_i)$$

Ovo nam dozvoljava da ograničimo odozdo logaritam sume, što se koristi u jednoj od verzija EM algoritma.

Definicija A.3 (Logaritamska vjerovatnoća - Log Likelihood). [3]

$$L(\theta) = \ln \mathcal{P}(\mathbf{X}|\theta)$$

Logaritamska vjerovatnoća predstavlja funkciju parametra θ ukoliko su dati podaci $\mathbf{X}$. Kako je $\ln(x)$ strogo rastuća funkcija, vrijednost θ koje maksimizira $\mathcal{P}(\mathbf{X}|\theta)$ isto tako maksimizira θ .

Implementacija algoritma

```
#ukljucivanje biblioteka

from scipy.stats import norm
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import hist

#potrebne funkcije koje sam napravila za EM algoritam:

def normalnaGustinaVjerovatnoceUTacki(xi, a, sigma):
    eksponent = -0.5 * ((xi - a) / sigma)**2
    fi = (sigma * np.sqrt(2 * np.pi))**(-1)
    fi = fi * np.exp(eksponent)
    return fi

def LogLikelihood(y, mi1, mi2, sigma1, sigma2, pi):
    N = np.size(y)
    LogLikelihood = 0
    for i in range(N):
        fi1 = normalnaGustinaVjerovatnoceUTacki(y[i], mi1, sigma1)
        fi2 = normalnaGustinaVjerovatnoceUTacki(y[i], mi2, sigma2)
        LogLikelihood = LogLikelihood + np.log((1 - pi) * fi1 + pi * fi2)/
            np.log(10)

    return LogLikelihood

def EM2GausAlgoritam(y, mi1poc, mi2poc, sigma1poc, sigma2poc, pipoc,
                    epsilon, maxIter):

    #postavljanje pocetnih parametara:
    mi1_hat = mi1poc; mi2_hat = mi2poc; sigma1_hat = sigma1poc; sigma2_hat
        = sigma2poc;

    pi_hat = pipoc

    #postavljanje dodatnih parametara i vrijednosti koje se vracaju iz
        funkcije:

    k = 0
    LL = []
    mi1_niz = []
    mi2_niz = []
    sigma1_niz = []
    sigma2_niz = []
    pi_niz = []
    N = np.size(y)

    ##Expectation step i Maximization step:
    while k < maxIter:

        #upisujemo sve varijable koje se mijenjaju kroz iteracije u
            odgovarajuće nizove:
```

```

mi1_niz.append(mi1_hat)
mi2_niz.append(mi2_hat)
sigma1_niz.append(sigma1_hat)
sigma2_niz.append(sigma2_hat)
pi_niz.append(pi_hat)

#odredjujemo log likelihood:
pomocna = LogLikelihood(y, mi1_hat, mi2_hat, sigma1_hat, sigma2_hat
                        , pi_hat)

LL.append(pomocna)

#E step:
gama_hat = []
for yi in y:
    fi1 = normalnaGustinaVjerovatnoceUTacki(yi, mi1_hat, sigma1_hat
                                             )
    fi2 = normalnaGustinaVjerovatnoceUTacki(yi, mi2_hat, sigma2_hat
                                             )
    gama_i = (pi_hat * fi2) / ((1 - pi_hat) * fi1 + pi_hat * fi2)
    gama_hat.append(gama_i)

#M step:
#prvo izracunamo sumu svih gami:
suma_gami = 0
for i in range(N):
    suma_gami = suma_gami + gama_hat[i]
#sad racunamo mi1, mi2, sigma1 i sigma2:
mi1 = 0; mi2 = 0; sigma1 = 0; sigma2 = 0;
for i in range(N):
    mi1 = mi1 + (1-gama_hat[i]) * y[i]
    mi2 = mi2 + gama_hat[i] * y[i]
mi1 = mi1 / (N - suma_gami)
mi2 = mi2 / suma_gami
for i in range(N):
    sigma1 = sigma1 + (1 - gama_hat[i]) * (y[i] - mi1)**2
    sigma2 = sigma2 + gama_hat[i] * (y[i] - mi2)**2
sigma1 = sigma1 / (N - suma_gami)
sigma2 = sigma2 / suma_gami
sigma1 = np.sqrt(sigma1)
sigma2 = np.sqrt(sigma2)
pi = suma_gami / N

#sada gledamo da li je algoritam konvergirao na osnovu log
likelihood-a:
if k > 1 and (LL[-1] - LL[-2])**2 < epsilon:
    break

#ako algoritam nije konvergirao, postavljamo vrijednosti parametara
za narednu iteraciju:
mi1_hat = mi1; mi2_hat = mi2; sigma1_hat = sigma1; sigma2_hat =
sigma2; pi_hat = pi;

k = k + 1

return mi1_hat, mi2_hat, sigma1_hat, sigma2_hat, pi_hat, k, LL, mi1_niz
, mi2_niz, sigma1_niz, sigma2_niz
, pi_niz

def PlotajIzvornuIDobivenuFunkciju(y, mi1, mi2, sigma1, sigma2, pi,
autobinovi = False):

N = 1000
x = np.linspace(np.min(y),np.max(y), N)
novaFunkcija = []
for i in range(N):

```

```

        fi1 = normalnaGustinaVjerovatnoceUTacki(x[i], mi1, sigma1)
        fi2 = normalnaGustinaVjerovatnoceUTacki(x[i], mi2, sigma2)
        novaFunkcija.append((1 - pi) * fi1 + pi * fi2)
#plotanje:
if autobinovi:
    plt.figure(); plt.hist(y, density = True); plt.plot(x, novaFunkcija
    );
    plt.legend(['Aproximirana distribucija pocetnog niza', 'Histogram
    pocetnog niza uzoraka'])
    return

binovi = int(np.sqrt(np.size(y)))
plt.figure(); plt.hist(y, bins = binovi, density = True); plt.plot(x,
    novaFunkcija);
plt.legend(['Aproximirana distribucija pocetnog niza', 'Histogram
    pocetnog niza uzoraka'])

```

```

#primjer iz knjige:

y1 = [-0.39, 0.12, 0.94, 1.67, 1.76, 2.44, 3.72, 4.28, 4.92, 5.53, 0.06, 0.
    48, 1.01, 1.68, 1.80, 3.25, 4.12, 4.
    60, 5.28, 6.22]

N = 20

#mi1 i mi2 nasumicno izaberemo iz niza y:
mi1poc = 0.94
mi2poc = 0.12

#sigma1 i sigma2 na pocetku mogu biti jednaki korijenu iz varijanse cijelog
    niza:
sigma1poc = np.sqrt(np.var(y1))
sigma2poc = sigma1poc

#pi_kapa moze biti na pocetku jednako 0.5
pipoc = 0.5

[mi1_hat, mi2_hat, sigma1_hat, sigma2_hat, pi_hat, k, LL, mi1_niz, mi2_niz,
    sigma1_niz, sigma2_niz, pi_niz] =
    EM2GausAlgoritam(y1, mi1poc, mi2poc,
    sigma1poc, sigma2poc, pipoc, 1e-10,
    100)

#novi parametri koje dobijemo su:
print('mi1_hat =', mi1_hat, 'mi2_hat =', mi2_hat)
print()
print('sigma1_hat^2 =', sigma1_hat**2, 'sigma2_hat^2 =', sigma2_hat**2)
print()
print('pi_hat =', pi_hat)
print()
print('Broj iteracija nakon kojih algoritam konvergira je', k+1)

PlotajIzvornuIDobivenuFunkciju(y1, mi1_hat, mi2_hat, sigma1_hat, sigma2_hat
    , pi_hat, autobinovi = True); plt.
    savefig('primjer1.png');

```

```

#plot log likelihood funkcije:
plt.figure(); plt.plot(range(0, k+1), LL); plt.title('Funkcija logaritamske
    vjerovatnoce za primjer 1'); plt.
    xlabel('Iteracija'); plt.savefig('
    primjer1_loglikelihood.png');

#kako se parametri mi1 i mi2 mijenjaju kroz iteracije:

```

```
plt.figure(); plt.plot(range(0, k+1), mi1_niz, marker = 'D'); plt.plot(
    range(0, k+1), mi2_niz, marker = 'x')
plt.xlabel('Iteracija'); plt.legend([' $\hat{\mu}_1$ ', ' $\hat{\mu}_2$ ']);
plt.title('Prikaz promjene parametara
 $\hat{\mu}_1$  i  $\hat{\mu}_2$  kroz
iteracije'); plt.savefig('mi_primjer1
.png');

#kako se sigma1 i sigma2 mijenjaju kroz iteracije:
plt.figure(); plt.plot(range(0, k+1), sigma1_niz, marker = 'D'); plt.plot(
    range(0, k+1), sigma2_niz, marker = '
x');
plt.xlabel('Iteracija'); plt.legend([' $\hat{\sigma}_1$ ', ' $\hat{\sigma}_2$ 
']); plt.title('Prikaz promjene
parametara  $\hat{\sigma}_1$  i  $\hat{\sigma}_2$ 
'); plt.savefig('
sigma_primjer1.png');

#kako se pi mijenja kroz iteracije:
plt.figure(); plt.plot(range(0, k+1), pi_niz, marker = 'D'); plt.xlabel('
Iteracija'); plt.title('Prikaz
promjene parametra  $\hat{\pi}$  kroz
iteracije'); plt.savefig('pi_primjer1
.png');
```

#moj primjer:

```
sigma1 = 5
sigma2 = 1
a1 = 20
a2 = 40
N1 = 4000
N2 = 6000

x1 = np.random.normal(a1, sigma1, N1)
x2 = np.random.normal(a2, sigma2, N2)
y = np.hstack((x1, x2))

np.random.shuffle(y)
```

```
mi1poc = y[0]
mi2poc = y[1]

sigma1poc = np.sqrt(np.var(y))
sigma2poc = sigma1poc

pipoc = 0.5

[mi1_hat, mi2_hat, sigma1_hat, sigma2_hat, pi_hat, k, LL, mi1_niz, mi2_niz,
    sigma1_niz, sigma2_niz, pi_niz] =
    EM2GausAlgoritam(y, mi1poc, mi2poc,
        sigma1poc, sigma2poc, pipoc, 1e-6,
        100)

#novi parametri koje dobijemo su:
print('mi1_hat =', mi1_hat, 'mi2_hat =', mi2_hat)
print()
print('sigma1_hat^2 =', sigma1_hat**2, 'sigma2_hat^2 =', sigma2_hat**2)
print()
print('pi_hat =', pi_hat)
print()
print('Broj iteracija je', k+1)
```

```
PlotajIzvornuIDobivenuFunkciju(y, mi1_hat, mi2_hat, sigma1_hat, sigma2_hat,
                                pi_hat); plt.savefig('primjer2.png')
;
```

```
#plot log likelihood funkcije:
plt.figure(); plt.plot(range(0, k+1), LL); plt.title('Funkcija logaritamske
vjerovatnoce za primjer 2'); plt.
xlabel('Iteracija'); plt.savefig('
primjer2_loglikelihood.png');

#kako se parametri mi1 i mi2 mijenjaju kroz iteracije:
plt.figure(); plt.plot(range(0, k+1), mi1_niz, marker = 'D'); plt.plot(
range(0, k+1), mi2_niz, marker = 'x')
;
plt.xlabel('Iteracija'); plt.legend([' $\hat{\mu}_1$ ', ' $\hat{\mu}_2$ ']);
plt.title('Prikaz promjene parametara
 $\hat{\mu}_1$  i  $\hat{\mu}_2$  kroz
iteracije'); plt.savefig('mi_primjer2
.png');

#kako se sigma1 i sigma2 mijenjaju kroz iteracije:
plt.figure(); plt.plot(range(0, k+1), sigma1_niz, marker = 'D'); plt.plot(
range(0, k+1), sigma2_niz, marker = '
x');
plt.xlabel('Iteracija'); plt.legend([' $\hat{\sigma}_1$ ', ' $\hat{\sigma}_2$ '
]); plt.title('Prikaz promjene
parametara  $\hat{\sigma}_1$  i  $\hat{\sigma}_2$ 
sigma_primjer2.png');

#kako se pi mijenja kroz iteracije:
plt.figure(); plt.plot(range(0, k+1), pi_niz, marker = 'D'); plt.xlabel('
Iteracija'); plt.title('Prikaz
promjene parametra  $\hat{\pi}$  kroz
iteracije'); plt.savefig('pi_primjer2
.png');
```

Bibliografija

- [1] T. Hastie, R. Tibshirani i J.H. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer series in statistics. Springer, 2009. ISBN: 9780387848846. URL: <https://books.google.ba/books?id=eBSgoAEACAAJ>.
- [2] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. Cambridge, MA, MIT Press, 1970 - 2012. ISBN: 9780262018029.
- [3] Sean Borman. "The expectation maximization algorithm-a short tutorial". *Submitted for publication* 41 (2004.).
- [4] A.R.V. J. N. Sharma. *Kirshna's Real Analysis: (General)*. Krishna Prakashan. URL: <https://books.google.ba/books?id=e27uJruMCBUC>.